



AMERICAN UNIVERSITY OF PHNOM PENH

STUDY LOCALLY. LIVE GLOBALLY.

COSC 251 – Fall 2026: Assignment 2 — Stacks and Queues (50 points)

Deadline: Oct 7, 11:59 PM

Prepared by: Visethboti Sin (v.sin@aupp.edu.kh)

Overview: In this assignment, you will implement the following classes: `Stack`, `Queue`, `PriorityQueue`, `StringReverser`, and `DelimiterChecker`.

When designing your solution, pay close attention to time complexity to earn full credit. The `Stack`, `Queue`, and `PriorityQueue` classes must use generics and adhere to the principles and functionalities covered in class.

Project Requirements:

1. The `Stack`, `Queue`, and `PriorityQueue` classes must be declared as generic classes: `Stack<T>`, `Queue<T>`, and `PriorityQueue<T>`.
2. The `Stack`, `Queue`, and `PriorityQueue` classes must be implemented from scratch using arrays.
3. If the internal array becomes full, the class should automatically resize the array.
4. If `pop()`, `remove()`, `peek()`, `peekFront()`, or `peekRear()` is called when the structure is empty, the method should return `null`.
5. Each required method must include a short comment stating its time complexity.
6. **Stack Class (LIFO)** Implement the following constructor and methods:
 - (a) `public Stack(int arraySize)`
 - (b) `public void push(T newItem)`
 - (c) `public T pop()`
 - (d) `public T peek()`
 - (e) `public String toString()`
 - (f) `public void display()`
7. **Queue Class (FIFO)** Implement the following constructor and methods:
 - (a) `public Queue(int arraySize)`
 - (b) `public void insert(T newItem)`
 - (c) `public T remove()`
 - (d) `public T peekFront()`
 - (e) `public T peekRear()`
 - (f) `public String toString()`
 - (g) `public void display()`

The `Queue` class may be implemented using a circular array or by shifting elements after removal.

8. PriorityQueue Class

Each item has an associated `priorityValue`. A smaller `priorityValue` means higher priority. Items must be removed based on priority.

Implement the following constructor and methods:

- (a) `public PriorityQueue(int arraySize)`
- (b) `public void insert(T newItem, int priorityValue)`
- (c) `public T remove()`
- (d) `public T peekFront()`
- (e) `public T peekRear()`
- (f) `public String toString()`
- (g) `public void display()`

9. **StringReverser Class**

Create a utility class that uses your **Stack** implementation and contains the following static method:

- (a) `public static String reverse(String input)`

This method accepts a string and returns its reversed version.

10. **DelimiterChecker Class**

Create a utility class that uses your **Stack** implementation and contains the following static method:

- (a) `public static boolean check(String input)`

This method validates the delimiters in a string and returns **true** if all delimiters match correctly. Otherwise, it should return **false**. It should only handle the following delimiters: `()`, `[]`, and `{}`.

What to Turn In: This is a group assignment; each group should submit only once.

1. Your **Stack** Java class with all required methods and functionality. The file must be named **Stack.java**, and the class must be named **Stack**.
2. Your **Queue** Java class with all required methods and functionality. The file must be named **Queue.java**, and the class must be named **Queue**.
3. Your **PriorityQueue** Java class with all required methods and functionality. The file must be named **PriorityQueue.java**, and the class must be named **PriorityQueue**.
4. Your **StringReverser** Java class with all required methods and functionality. The file must be named **StringReverser.java**, and the class must be named **StringReverser**.
5. Your **DelimiterChecker** Java class with all required methods and functionality. The file must be named **DelimiterChecker.java**, and the class must be named **DelimiterChecker**.
6. Optional: any additional supporting Java classes.
7. Note: Do not submit a **main** class. The submitted classes must not include a **main** method.

Grading:

1. Program Design (10): The overall quality of your solution design.
2. Program Execution (15): The program runs correctly without errors.
3. Specification Satisfaction (15): The program fully meets the given requirements.
4. Coding Style and Readability (10): Code is well-formatted, clear, and includes meaningful comments.